

Weather Information Service

Refactoring, Interfaces, Dependency Injection, CLI Design, and Testing

Due	October 4, 2026 — End of Day
Teams	Individual or pairs (maximum of 2 students)
Late Policy	May be submitted up to one day late for a 10% penalty

1. Project Overview

You are provided with starter code for a small Java 25 application that retrieves current temperature information from the Open-Meteo weather service and displays it for several predefined cities. The starter program works, but it combines several responsibilities in one source file: location configuration, HTTP communication, response parsing, error handling, application control, and presentation.

Your task is to refactor the prototype into a well-designed, testable, extensible command-line weather information system. The goal is not simply to add features. Your design should demonstrate the software design principles discussed in class.

Central Design Question

If the real weather API were replaced with a completely different source of weather data, how much of your program would you have to rewrite?

A strong design should require changes primarily to the provider implementation—not to the command-line interface, application logic, or unit tests for that logic.

2. Learning Objectives

- Separate responsibilities across classes and packages.
- Program to interfaces rather than concrete implementations.
- Use constructor-based dependency injection.
- Use composition and encapsulation to create maintainable components.
- Design code that can be unit tested without an Internet connection.
- Use JUnit Jupiter and controlled test implementations to isolate external data access.
- Create a usable command-line interface (CLI).
- Use appropriate modern Java 25 constructs.
- Handle errors explicitly rather than silently substituting incorrect data.

3. Working Individually or in Pairs

You may complete this project individually or with one partner. A team may contain no more than two students. Students working as a pair should submit one project and both students must be identified in the README. Both partners are responsible for understanding the complete design and implementation, not only the portion they personally wrote.

4. Due Date and Late Policy

The project is due October 4, 2026, by the end of the day.

A submission may be turned in up to one day late for a 10% penalty. Submissions more than one day late are subject to the course's normal late-work policy.

5. Starter Application

The supplied `Main.java` defines several locations, creates an `HttpClient`, builds and sends a request to Open-Meteo, extracts temperature data from the JSON response, and renders a simple text bar.

The starter code should be treated as a prototype to refactor, not as the architecture of your final solution. In particular, your final design must separate networking, weather-data access, application logic, command processing, and presentation.

Important: the starter parser returns 72.0 when a temperature cannot be found. Your refactored application must not silently replace missing or malformed data with a believable fake value. Parsing and provider failures must be handled explicitly.

6. Weather API and Data Research

The starter code uses the Open-Meteo Forecast API. You are expected to examine the API documentation and determine which additional weather measurements are useful for your application.

Open-Meteo Forecast API documentation: <https://open-meteo.com/en/docs>

The API supports current, hourly, and daily weather variables. Examples include relative humidity, apparent temperature, precipitation, weather codes, pressure, cloud cover, wind speed, wind direction, and wind gusts. Research the documentation rather than limiting your implementation to the temperature field used in the starter code.

7. Required Architecture

Your solution must use the package and source organization shown in Section 8. You may add additional classes when they improve the design, but you may not collapse the required responsibilities into fewer classes.

Weather Data Provider

Create a `WeatherDataProvider` interface that represents a source of weather information. Application logic must request weather information through this abstraction rather than directly through `HttpClient`.

Create `OpenMeteoWeatherProvider` as the concrete production implementation. It is responsible for communicating with Open-Meteo and converting API responses into your application's model objects.

Weather Service

Create `WeatherService` to contain weather-related application logic. `WeatherService` must receive a `WeatherDataProvider` through its constructor. It must not construct `OpenMeteoWeatherProvider` internally.

```
WeatherService → WeatherDataProvider ← OpenMeteoWeatherProvider
```

This constructor-based dependency injection must make it possible to supply a mock or test implementation during unit testing.

Weather Data Model

Use records/classes in the model package to represent locations and weather information. Raw JSON strings must not be passed through the rest of the application.

Command-Line Interface

WeatherCLI is responsible for reading commands, validating input, calling service methods, and displaying results. It must not directly communicate with the remote API. Service classes should return information rather than print directly to the console.

Application Startup

Main.java is the composition root: it should create the production provider, construct WeatherService with that provider, construct the CLI, and start the application.

8. Mandatory Project Organization

The following organization is required. You may add files and subcomponents, but these packages and required classes must be present and used for their stated responsibilities.

```
src/
├── main/
│   ├── java/
│   │   └── weather/
│   │       ├── Main.java
│   │       ├── model/
│   │       │   ├── Location.java
│   │       │   └── WeatherData.java
│   │       ├── provider/
│   │       │   ├── WeatherDataProvider.java
│   │       │   └── OpenMeteoWeatherProvider.java
│   │       ├── service/
│   │       │   └── WeatherService.java
│   │       └── cli/
│   │           └── WeatherCLI.java
└── test/
    ├── java/
    │   └── weather/
    │       ├── WeatherServiceTest.java
    │       └── WeatherCLITest.java
```

Required test files: WeatherServiceTest.java must include at least 8 meaningful unit tests, and WeatherCLITest.java must include at least 5 meaningful unit tests.

9. Required Command-Line Functionality

When the application starts, users must be able to enter commands interactively. You may make reasonable improvements to the command syntax, but the following functionality is required.

```
Weather Information Service
```

```

weather> help
weather> current Chicago
weather> compare Chicago "New York"
weather> summary Chicago
weather> locations
weather> quit

```

help — Display the available commands and their syntax.

locations — Display the locations currently known by the application. At minimum, support Chicago, Los Angeles, and New York.

current <location> — Display the location, current temperature, and at least two additional current-weather measurements selected from the API.

compare <location1> <location2> — Compare current weather for two locations. At minimum, compare temperature and the additional measurements your application supports.

summary <location> — Produce a readable sentence or short paragraph summarizing current conditions.

quit — Exit the application cleanly.

10. Additional Functionality

Implement at least one meaningful additional feature that uses the existing architecture. Examples include finding the warmest/coldest configured location, comparing all locations, converting units, displaying a richer ASCII visualization, identifying the strongest wind, or using forecast data. You may propose another weather-related feature.

The feature must use your interfaces and service layer rather than bypassing them.

11. Dependency Injection Requirements

Objects that need weather data must receive a `WeatherDataProvider` instead of constructing a real provider themselves. `Main.java` should assemble the production objects. This is a required design feature, not an optional enhancement.

```

Main
|
+-- OpenMeteoWeatherProvider
|
+-- WeatherService(WeatherDataProvider)
|
+-- WeatherCLI(WeatherService)

```

12. Unit Testing

Use JUnit Jupiter (JUnit 5) for automated testing. Design your code so tests can supply controlled test implementations of `WeatherDataProvider` instead of contacting the real data source. Automated unit tests must not contact Open-Meteo or require an Internet connection.

Your tests must cover:

- Retrieving current weather through WeatherService.
- Correct handling of known locations.
- Correct handling of unknown locations.
- Weather comparison logic.
- Summary-generation logic.
- The additional feature you implemented.
- Invalid command handling.
- At least one failure returned by WeatherDataProvider.

Tests should verify behavior with assertions rather than simply printing output. Use controlled test implementations of WeatherDataProvider to demonstrate how dependency injection makes the service independently testable.

Minimum test counts: WeatherServiceTest.java must contain at least 8 meaningful JUnit test methods, and WeatherCLITest.java must contain at least 5 meaningful JUnit test methods. These are minimums; additional tests may be necessary to adequately test your implementation. Each test should verify a distinct behavior or case rather than duplicating another test with trivial changes.

13. Java 25 Suggestions

Java 25 provides several language features that may make parts of this project clearer or more concise. You are encouraged to consider these features where they fit naturally, but you are not required to use a specific number of them.

- Records — useful for immutable model objects such as Location or WeatherData.
- Enhanced switch expressions — useful for interpreting CLI commands or mapping values to categories.
- Pattern matching — useful when it simplifies type checks or command/result handling.
- Text blocks — useful for a multi-line help screen, usage instructions, or formatted output.
- var — useful for local variables when the type is obvious and readability is preserved.
- Modern collection APIs and streams — useful for filtering locations, finding the warmest/coldest location, calculating comparisons, or formatting results.
- Modern instanceof patterns — useful when type checking is actually needed and avoids unnecessary casts.

Use modern Java features because they improve the design or readability of your solution, not simply to demonstrate syntax. Threads and concurrency have not been covered and must not be used in this project.

14. Error Handling

Your application must fail gracefully and provide useful feedback. Consider unknown commands, missing arguments, unknown locations, network failures, unsuccessful HTTP status codes, malformed/missing API data, and invalid weather values.

Do not silently substitute fake data when parsing or communication fails. Choose an appropriate error-handling strategy and make failures visible to the appropriate layer.

15. Separation of Concerns

- CLI code must not know how HTTP communication works.
- WeatherService must not print directly to the console.
- HTTP/provider code must not decide how results are displayed.
- Parsing/provider code must not contain CLI control logic.
- Unit tests must not require Internet access.
- Main should assemble dependencies rather than contain application logic.

16. Maven Requirements

The submission must be a Maven project configured for Java 25. The pom.xml must include JUnit Jupiter and the approved mocking library used by the project.

All tests must run successfully with:

```
mvn test
```

The project must compile and run from a clean checkout without depending on IDE-specific configuration.

17. README Requirements

Include README.md with the following sections:

- Team Members — list both students if working in a pair.
- Running the Application — how to compile and run the program.
- Running the Tests — how to execute the complete test suite.
- Design — major components and their responsibilities.
- Interfaces — where interfaces are used and why.
- Dependency Injection — identify the injected dependency and explain the benefit.
- Testing — explain how the application is tested without the live API.
- Java 25 — if you used modern Java 25 features, identify them and briefly explain where they helped the design or readability.
- Additional Feature — describe the additional functionality you implemented.

Design Reflection Questions

1. What responsibilities were combined in the original starter code that you separated during refactoring?
2. How does programming to WeatherDataProvider make the program easier to change?
3. How does dependency injection make WeatherService easier to test?
4. Why should unit tests avoid depending on the live weather API?
5. If the current weather API were replaced with a completely different provider, which parts of your application would need to change? Which parts should not need to change?

18. Required Deliverables

- Complete Maven project.
- All Java source files using the mandatory package organization.
- All JUnit tests.
- pom.xml.
- README.md containing the required documentation and reflection answers.

19. Restrictions

- Do not use threads, ExecutorService, asynchronous HTTP requests, or CompletableFuture.
- Do not create a GUI.
- Do not place the entire solution in one class.
- Do not make real API requests from unit tests.
- Do not hard-code test-specific behavior into production classes.
- Do not bypass WeatherDataProvider from the CLI or service layer.